

How to (kind of) Win Monopoly

M325K

Saksham Agarwal

University of Texas at Austin

April 13, 2026

Outline

- 1 Motivation
- 2 Setting Up the Math
- 3 Simulating Turns
- 4 Finding the Equilibrium
- 5 Why Does This Work?
- 6 A Better Approach
- 7 The Best Color Set

Motivation

Growing up, my family played a lot of board games. Monopoly was one of them.

Motivation

Growing up, my family played a lot of board games. Monopoly was one of them.

My favorite color set was dark blue, but it rarely worked.

Motivation

Growing up, my family played a lot of board games. Monopoly was one of them.

My favorite color set was dark blue, but it rarely worked.

It was too expensive, and barely anyone landed on the squares. By the time someone did, I had already mortgaged the property.

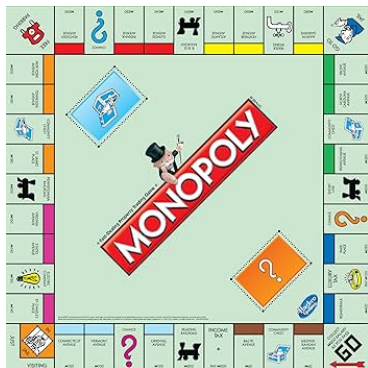
Motivation

Growing up, my family played a lot of board games. Monopoly was one of them.

My favorite color set was dark blue, but it rarely worked.

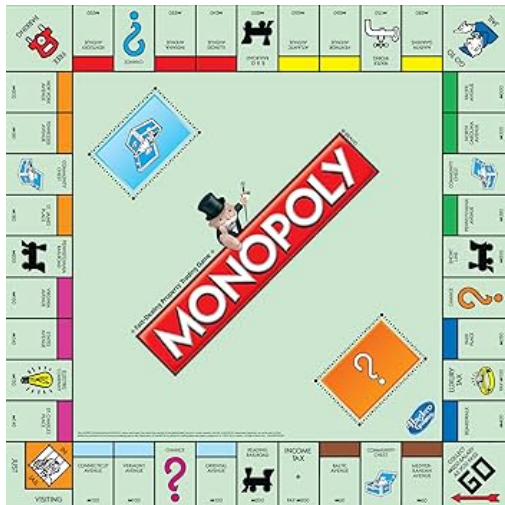
It was too expensive, and barely anyone landed on the squares. By the time someone did, I had already mortgaged the property.

Monopoly is supposed to be a game of chance. But I started to wonder if there might be an **optimal strategy** hiding under the randomness, and that this could be a **math problem** in disguise.



Source: TurboSquid, by Shutterstock

The Board



What Is a Directed Graph?

Definition

A **directed graph** is a type of graph, G , where edges have a direction associated with them. In other words, $G = (V, E)$ where:

- V is a finite set of **vertices** (nodes)
- E is a set of **directed edges**, ordered pairs (i, j) with $i, j \in V$

What Is a Directed Graph?

Definition

A **directed graph** is a type of graph, G , where edges have a direction associated with them. In other words, $G = (V, E)$ where:

- V is a finite set of **vertices** (nodes)
- E is a set of **directed edges**, ordered pairs (i, j) with $i, j \in V$

An edge $(i, j) \in E$ means there is a connection **from** i **to** j .

What Is a Directed Graph?

Definition

A **directed graph** is a type of graph, G , where edges have a direction associated with them. In other words, $G = (V, E)$ where:

- V is a finite set of **vertices** (nodes)
- E is a set of **directed edges**, ordered pairs (i, j) with $i, j \in V$

An edge $(i, j) \in E$ means there is a connection **from** i **to** j .

Order matters: $(i, j) \neq (j, i)$, for the most part.

What Is a Directed Graph?

Definition

A **directed graph** is a type of graph, G , where edges have a direction associated with them. In other words, $G = (V, E)$ where:

- V is a finite set of **vertices** (nodes)
- E is a set of **directed edges**, ordered pairs (i, j) with $i, j \in V$

An edge $(i, j) \in E$ means there is a connection **from** i **to** j .

Order matters: $(i, j) \neq (j, i)$, for the most part.

Indegree: incoming edges into a vertex V

Outdegree: edges going out of a vertex V

The Board as a Directed Graph

$G = (V, E)$ with $V = \{0, 1, \dots, 39\}$.

The Board as a Directed Graph

$G = (V, E)$ with $V = \{0, 1, \dots, 39\}$.

{Go: 0, Jail: 10, Free Parking: 20, Go to Jail: 30}

The Board as a Directed Graph

$G = (V, E)$ with $V = \{0, 1, \dots, 39\}$.

{Go: 0, Jail: 10, Free Parking: 20, Go to Jail: 30}

In code, using a dictionary is easy to store key-value pairs. Here, it's to save space on this slide.

The Board as a Directed Graph

$G = (V, E)$ with $V = \{0, 1, \dots, 39\}$.

{Go: 0, Jail: 10, Free Parking: 20, Go to Jail: 30}

In code, using a dictionary is easy to store key-value pairs. Here, it's to save space on this slide.

Edge $(i, j) \in E$ if a player on square i can reach j in one turn.

The Board as a Directed Graph

$G = (V, E)$ with $V = \{0, 1, \dots, 39\}$.

{Go: 0, Jail: 10, Free Parking: 20, Go to Jail: 30}

In code, using a dictionary is easy to store key-value pairs. Here, it's to save space on this slide.

Edge $(i, j) \in E$ if a player on square i can reach j in one turn.

Most vertices: out-degree 11 (dice sums 2 through 12).

Exceptions:

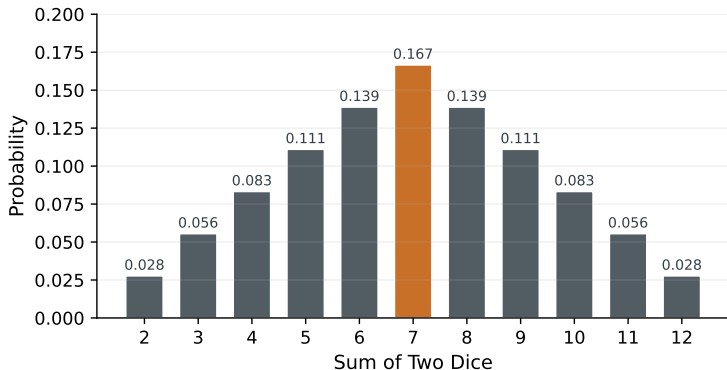
- **Go To Jail** (vertex 30): out-degree 1
- **Chance** (vertices 7, 22, 36): out-degree > 11 (card destinations)
- **Community Chest** (vertices 2, 17, 33): cards redirect to GO or Jail

Weighting the Edges

The probability distribution of the sum of two dice is not uniform. For instance, rolling a 7 is six times more likely than rolling a 2.

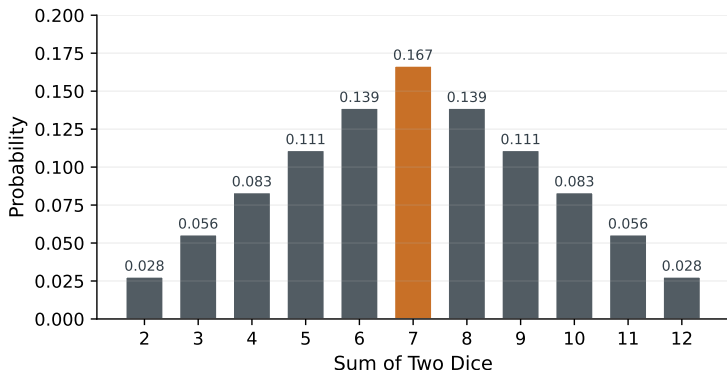
Weighting the Edges

The probability distribution of the sum of two dice is not uniform. For instance, rolling a 7 is six times more likely than rolling a 2.



Weighting the Edges

The probability distribution of the sum of two dice is not uniform. For instance, rolling a 7 is six times more likely than rolling a 2.



So, the edges of the directed graph won't be evenly weighted; some will matter more than others.

What Is a Transition Matrix?

Definition

A **transition matrix** $\mathbf{P}$ is a square matrix where entry P_{ij} is the probability of moving from state i to state j :

$$P_{ij} = \Pr(\text{next state} = j \mid \text{current state} = i)$$

with $P_{ij} \geq 0$ and $\sum_j P_{ij} = 1$ for every row i .

What Is a Transition Matrix?

Definition

A **transition matrix** $\mathbf{P}$ is a square matrix where entry P_{ij} is the probability of moving from state i to state j :

$$P_{ij} = \Pr(\text{next state} = j \mid \text{current state} = i)$$

with $P_{ij} \geq 0$ and $\sum_j P_{ij} = 1$ for every row i .

A matrix with these properties is also called a **stochastic matrix**.

What Is a Transition Matrix?

Definition

A **transition matrix** $\mathbf{P}$ is a square matrix where entry P_{ij} is the probability of moving from state i to state j :

$$P_{ij} = \Pr(\text{next state} = j \mid \text{current state} = i)$$

with $P_{ij} \geq 0$ and $\sum_j P_{ij} = 1$ for every row i .

A matrix with these properties is also called a **stochastic matrix**.

Each row is a probability distribution over all possible next moves, starting at that square.

The Transition Matrix $\mathbf{P}$

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & \frac{1}{36} & \frac{2}{36} & \cdots & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{36} & \cdots & 0 & 0 \\ \vdots & & & & \ddots & & \vdots \\ 0 & \cdots & & 1 & & \cdots & 0 \\ \vdots & & & & \ddots & & \vdots \\ \frac{1}{36} & \frac{2}{36} & \frac{3}{36} & \frac{4}{36} & \cdots & 0 & 0 \\ 0 & \frac{1}{36} & \frac{2}{36} & \frac{3}{36} & \cdots & 0 & 0 \end{pmatrix}_{40 \times 40}$$

The Transition Matrix $\mathbf{P}$

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & \frac{1}{36} & \frac{2}{36} & \cdots & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{36} & \cdots & 0 & 0 \\ \vdots & & & & \ddots & & \vdots \\ 0 & \cdots & & 1 & & \cdots & 0 \\ \vdots & & & & \ddots & & \vdots \\ \frac{1}{36} & \frac{2}{36} & \frac{3}{36} & \frac{4}{36} & \cdots & 0 & 0 \\ 0 & \frac{1}{36} & \frac{2}{36} & \frac{3}{36} & \cdots & 0 & 0 \end{pmatrix}_{40 \times 40}$$

The bottom rows "wrap around" the board, with a cycle of modulus 40.

Assumptions & Simplifications

- No doubles

Assumptions & Simplifications

- No doubles
- Single player

Assumptions & Simplifications

- No doubles
- Single player
- Chance/Community Chest redirects are slightly delayed (but probabilistically insignificant), as I treated landing on Chance and drawing a card as two steps, not one

Assumptions & Simplifications

- No doubles
- Single player
- Chance/Community Chest redirects are slightly delayed (but probabilistically insignificant), as I treated landing on Chance and drawing a card as two steps, not one
- No “Get Out of Jail Free” card tracking, or paying \$50 to get out. You have to spend 3 turns.

Assumptions & Simplifications

- No doubles
- Single player
- Chance/Community Chest redirects are slightly delayed (but probabilistically insignificant), as I treated landing on Chance and drawing a card as two steps, not one
- No “Get Out of Jail Free” card tracking, or paying \$50 to get out. You have to spend 3 turns.

Technically, none of these assumptions are necessary, but it makes the math much simpler, for what will end up being a roughly similar distribution.

The Markov Property

Where you go next depends only on where you are now:

$$\Pr(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = P_{ij}$$

The Markov Property

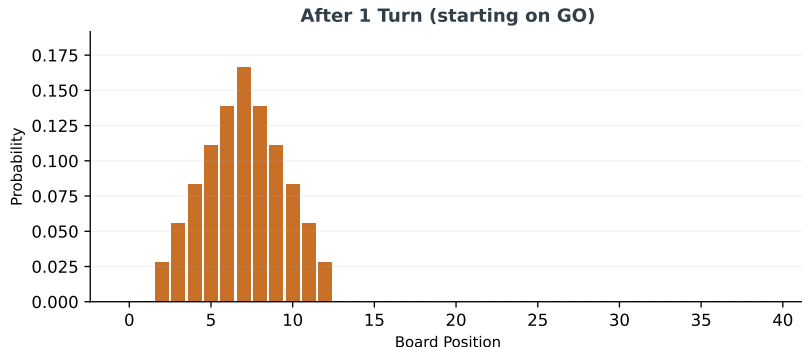
Where you go next depends only on where you are now:

$$\Pr(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = P_{ij}$$

On Tennessee Avenue, your next move is the same. Regardless of if you came from Jail or Pennsylvania Railroad, P_{ij} is only dependent on your current position.

After 1 Turn

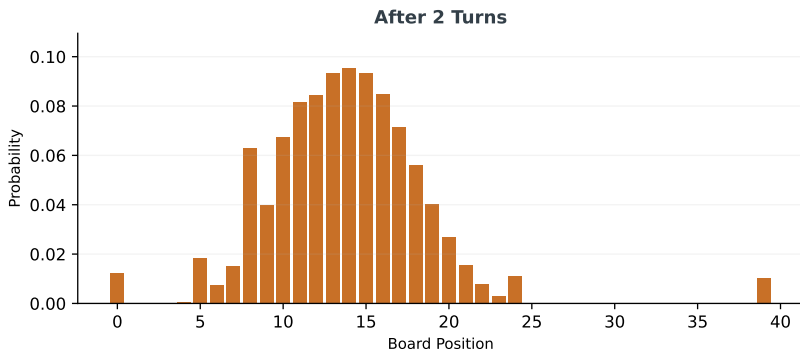
Start on Go: $\mathbf{v}^0 = (1, 0, 0, \dots, 0)$. Multiplying by $\mathbf{P}$ gives:



Probability you land on squares 2 through 12. Basically the dice distribution from earlier, but a little shifted.

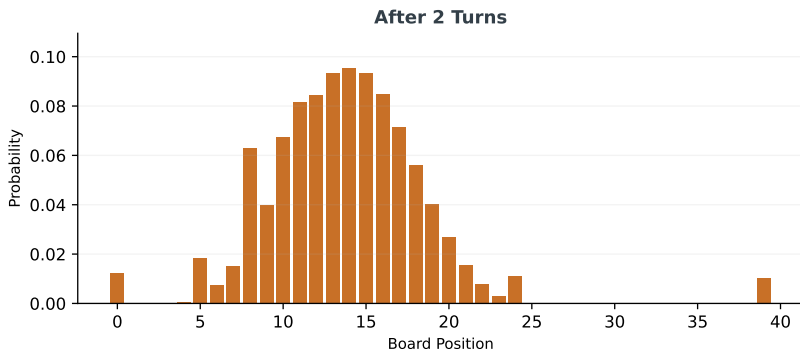
After 2 Turns

Multiply again: $\mathbf{v}^{(2)} = \mathbf{v}^{(1)}\mathbf{P}$.



After 2 Turns

Multiply again: $\mathbf{v}^{(2)} = \mathbf{v}^{(1)}\mathbf{P}$.



The distribution is less shaped like a dice roll, and more "random"

But What Happens in the Long Run?

Each multiplication $\mathbf{v}^{(n+1)} = \mathbf{v}^{(n)}\mathbf{P}$ simulates one more turn.

But What Happens in the Long Run?

Each multiplication $\mathbf{v}^{(n+1)} = \mathbf{v}^{(n)}\mathbf{P}$ simulates one more turn.

Does the distribution keep changing forever, or does it **settle down** eventually?

But What Happens in the Long Run?

Each multiplication $\mathbf{v}^{(n+1)} = \mathbf{v}^{(n)}\mathbf{P}$ simulates one more turn.

Does the distribution keep changing forever, or does it **settle down** eventually?

We want π such that:

$$\pi\mathbf{P} = \pi, \quad \sum_i \pi_i = 1$$

π : The row vector showing the probability distribution over time.

But What Happens in the Long Run?

Each multiplication $\mathbf{v}^{(n+1)} = \mathbf{v}^{(n)}\mathbf{P}$ simulates one more turn.

Does the distribution keep changing forever, or does it **settle down** eventually?

We want π such that:

$$\pi\mathbf{P} = \pi, \quad \sum_i \pi_i = 1$$

π : The row vector showing the probability distribution over time.

π becomes a **stationary distribution** when we multiply it by $\mathbf{P}$ and (basically) nothing changes in π .

But What Happens in the Long Run?

Each multiplication $\mathbf{v}^{(n+1)} = \mathbf{v}^{(n)}\mathbf{P}$ simulates one more turn.

Does the distribution keep changing forever, or does it **settle down** eventually?

We want π such that:

$$\pi\mathbf{P} = \pi, \quad \sum_i \pi_i = 1$$

π : The row vector showing the probability distribution over time.

π becomes a **stationary distribution** when we multiply it by $\mathbf{P}$ and (basically) nothing changes in π .

Approach 1: Just keep multiplying until the distribution stops changing. We'll track change via a **loss function** to know when to stop.

Power Iteration

Let $\mathbf{v}^{(n)}$ be a probability vector over the 40 squares at turn n .

Power Iteration

Let $\mathbf{v}^{(n)}$ be a probability vector over the 40 squares at turn n .

Algorithm:

$$\mathbf{v}^0 = \left(\frac{1}{40}, \dots, \frac{1}{40} \right), \quad \mathbf{v}^{(n+1)} = \mathbf{v}^{(n)} \mathbf{P}$$

Assume a uniform distribution for $\mathbf{v}^0$

Power Iteration

Let $\mathbf{v}^{(n)}$ be a probability vector over the 40 squares at turn n .

Algorithm:

$$\mathbf{v}^0 = \left(\frac{1}{40}, \dots, \frac{1}{40} \right), \quad \mathbf{v}^{(n+1)} = \mathbf{v}^{(n)} \mathbf{P}$$

Assume a uniform distribution for $\mathbf{v}^0$

Loss function: measure how much the distribution changed this step.

$$L(n) = \left\| \mathbf{v}^{(n+1)} - \mathbf{v}^{(n)} \right\|$$

Power Iteration

Let $\mathbf{v}^{(n)}$ be a probability vector over the 40 squares at turn n .

Algorithm:

$$\mathbf{v}^0 = \left(\frac{1}{40}, \dots, \frac{1}{40}\right), \quad \mathbf{v}^{(n+1)} = \mathbf{v}^{(n)} \mathbf{P}$$

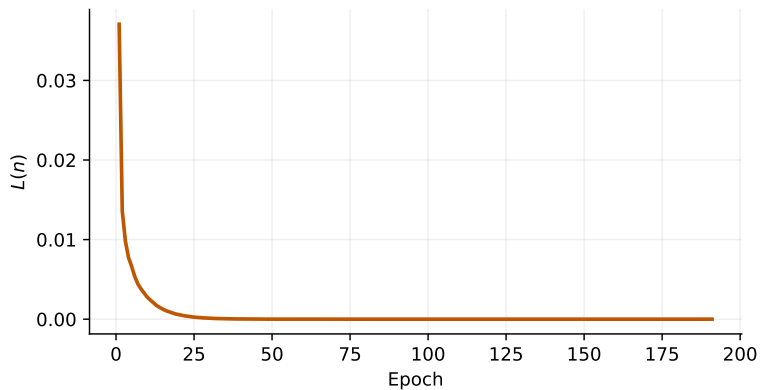
Assume a uniform distribution for $\mathbf{v}^0$

Loss function: measure how much the distribution changed this step.

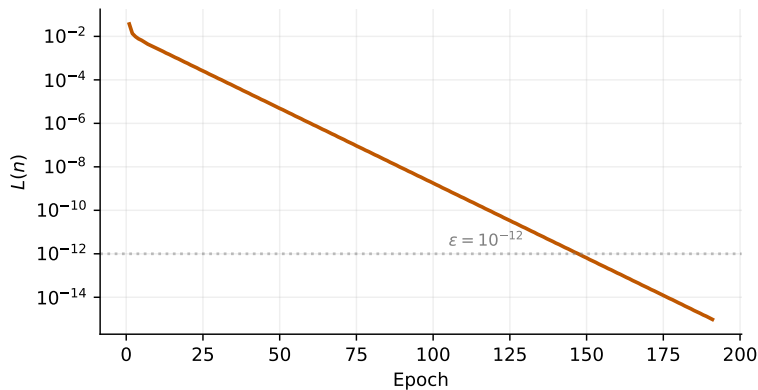
$$L(n) = \left\| \mathbf{v}^{(n+1)} - \mathbf{v}^{(n)} \right\|$$

Stop when $L(n) < \varepsilon$. This is very similar to how machine learning models are trained, but used in a slightly different context. Note that epsilon is just some random small number we can choose. I chose to ignore it when I was implementing code, but it's good practice not to.

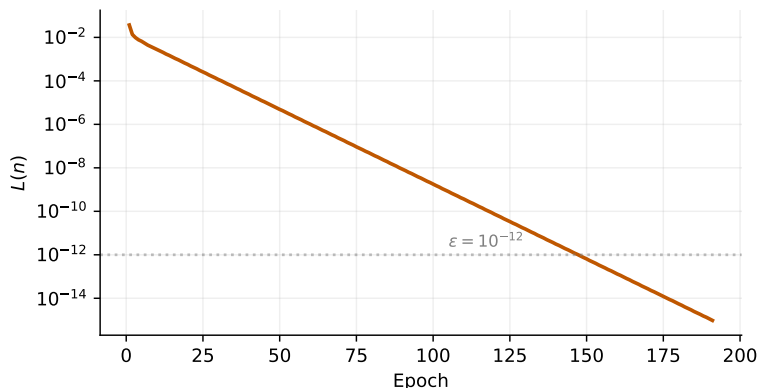
The Loss Curve



The Loss Curve (Log Scale)

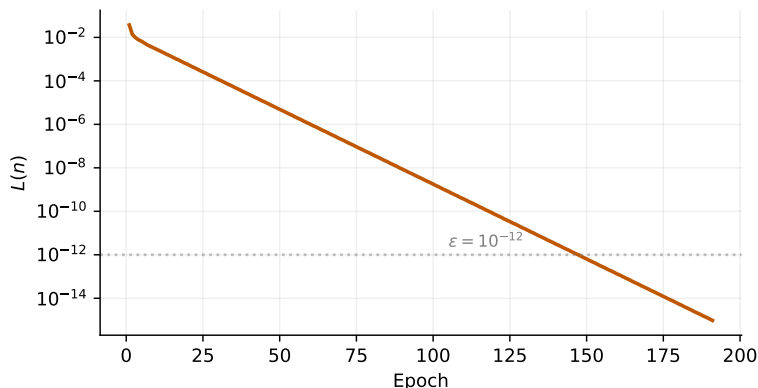


The Loss Curve (Log Scale)



$L(n)$ gets really close to 0, but not quite. But the difference is less than a trillionth, so we can say we've essentially found the stationary distribution.

The Loss Curve (Log Scale)



$L(n)$ gets really close to 0, but not quite. But the difference is less than a trillionth, so we can say we've essentially found the stationary distribution.

Converged in **191 iterations**. Final loss: 9.58×10^{-16} .

A Quick Tangent

We computed $\mathbf{v}^{(n)} = \mathbf{v}^{(0)}\mathbf{P}^n$ by multiplying a vector by a matrix over and over again. Shouldn't everything shrink to zero?

A Quick Tangent

We computed $\mathbf{v}^{(n)} = \mathbf{v}^{(0)}\mathbf{P}^n$ by multiplying a vector by a matrix over and over again. Shouldn't everything shrink to zero?

Yes.

A Quick Tangent

We computed $\mathbf{v}^{(n)} = \mathbf{v}^{(0)}\mathbf{P}^n$ by multiplying a vector by a matrix over and over again. Shouldn't everything shrink to zero?

Yes. But $\mathbf{P}$ is special.

Stochastic Matrices Preserve Energy

Theorem

If $\mathbf{P}$ is a stochastic matrix and $\mathbf{v}$ is a probability distribution (i.e. $v_i \geq 0$ and $\sum_i v_i = 1$), then $\mathbf{vP}$ is also a probability distribution.

Stochastic Matrices Preserve Energy (Proof)

Proof.

Let $\mathbf{P}$ be stochastic and $\mathbf{v}$ a probability distribution. Consider $\sum_j (\mathbf{vP})_j$. We wish to show that this product is also a probability distribution.

Stochastic Matrices Preserve Energy (Proof)

Proof.

Let $\mathbf{P}$ be stochastic and $\mathbf{v}$ a probability distribution. Consider $\sum_j (\mathbf{vP})_j$. We wish to show that this product is also a probability distribution. Expanding this product and swapping the order of summation since the matrices are finite gives us,

$$\begin{aligned}\sum_j (\mathbf{vP})_j &= \sum_j \sum_i v_i P_{ij}, \\ &= \sum_i v_i \sum_j P_{ij}.\end{aligned}$$



Stochastic Matrices Preserve Energy (Proof, cont.)

Proof.

Since $\mathbf{P}$ is stochastic, $\sum_j P_{ij} = 1$. Substituting,

$$\begin{aligned} &= \sum_i v_i \cdot 1, \\ &= \sum_i v_i, \\ &= 1. \end{aligned}$$

Therefore $\sum_j (\mathbf{vP})_j = 1$, so $\mathbf{vP}$ is also a probability distribution. □

So It Can't Decay

This means the entries (or rows) of $\mathbf{v}^{(n)}$ always sum to 1, no matter how many times we multiply.

So It Can't Decay

This means the entries (or rows) of $\mathbf{v}^{(n)}$ always sum to 1, no matter how many times we multiply.

Energy is **conserved**. Each step redistributes it across the 40 squares, but never creates or destroys any.

So It Can't Decay

This means the entries (or rows) of $\mathbf{v}^{(n)}$ always sum to 1, no matter how many times we multiply.

Energy is **conserved**. Each step redistributes it across the 40 squares, but never creates or destroys any.

This also makes intuitive sense. Since $\mathbf{v}$ is the probability distribution, all probabilities must sum to 1. You can't go nowhere.

So It Can't Decay

This means the entries (or rows) of $\mathbf{v}^{(n)}$ always sum to 1, no matter how many times we multiply.

Energy is **conserved**. Each step redistributes it across the 40 squares, but never creates or destroys any.

This also makes intuitive sense. Since $\mathbf{v}$ is the probability distribution, all probabilities must sum to 1. You can't go nowhere.

But why only *one specific* distribution?

π Is an Eigenvector

Our stationary condition:

$$\pi \mathbf{P} = \mathbf{1} \cdot \pi$$

π Is an Eigenvector

Our stationary condition:

$$\pi \mathbf{P} = 1 \cdot \pi$$

π is an eigenvector of $\mathbf{P}$ with eigenvalue $\lambda = 1$.

π Is an Eigenvector

Our stationary condition:

$$\pi \mathbf{P} = 1 \cdot \pi$$

π is an eigenvector of $\mathbf{P}$ with eigenvalue $\lambda = 1$.

Instead of iterating 191 times, we can find π **directly** by computing the eigenvector. Plus, we get an exact answer compared to a good estimate.

π Is an Eigenvector

Our stationary condition:

$$\pi \mathbf{P} = 1 \cdot \pi$$

π is an eigenvector of $\mathbf{P}$ with eigenvalue $\lambda = 1$.

Instead of iterating 191 times, we can find π **directly** by computing the eigenvector. Plus, we get an exact answer compared to a good estimate.

I never computed eigenvectors by hand. That would take too long. I had NumPy do it. However,

The Transpose Trick

NumPy only solves $\mathbf{P}\boldsymbol{\pi} = \lambda\boldsymbol{\pi}$ (right eigenvectors). We have $\boldsymbol{\pi}\mathbf{P} = \lambda\boldsymbol{\pi}$ (left eigenvector).

The Transpose Trick

NumPy only solves $\mathbf{P}\pi = \lambda\pi$ (right eigenvectors). We have $\pi\mathbf{P} = \lambda\pi$ (left eigenvector).

Theorem

π is a left eigenvector of $\mathbf{P}$ with eigenvalue λ if and only if $\pi^\top$ is a right eigenvector of $\mathbf{P}^\top$ with the same eigenvalue λ .

The Transpose Trick

NumPy only solves $\mathbf{P}\pi = \lambda\pi$ (right eigenvectors). We have $\pi\mathbf{P} = \lambda\pi$ (left eigenvector).

Theorem

π is a left eigenvector of $\mathbf{P}$ with eigenvalue λ if and only if $\pi^\top$ is a right eigenvector of $\mathbf{P}^\top$ with the same eigenvalue λ .

Let's prove it.

The Transpose Trick (Proof)

Proof.

We prove the forward direction. Suppose $\pi \mathbf{P} = \lambda \pi$.

The Transpose Trick (Proof)

Proof.

We prove the forward direction. Suppose $\pi \mathbf{P} = \lambda \pi$.

Transposing both sides,

$$(\pi \mathbf{P})^{\top} = (\lambda \pi)^{\top}$$

The Transpose Trick (Proof)

Proof.

We prove the forward direction. Suppose $\pi \mathbf{P} = \lambda \pi$.

Transposing both sides,

$$(\pi \mathbf{P})^\top = (\lambda \pi)^\top$$

Applying $(AB)^\top = B^\top A^\top$ on the left side,

$$\mathbf{P}^\top \pi^\top = \lambda \pi^\top$$

The Transpose Trick (Proof)

Proof.

We prove the forward direction. Suppose $\pi \mathbf{P} = \lambda \pi$.

Transposing both sides,

$$(\pi \mathbf{P})^\top = (\lambda \pi)^\top$$

Applying $(AB)^\top = B^\top A^\top$ on the left side,

$$\mathbf{P}^\top \pi^\top = \lambda \pi^\top$$

This is a right eigenvector equation for $\mathbf{P}^\top$. The reverse direction can be proved similarly by applying the transposing back. □

The Transpose Trick (Proof)

Proof.

We prove the forward direction. Suppose $\pi \mathbf{P} = \lambda \pi$.

Transposing both sides,

$$(\pi \mathbf{P})^\top = (\lambda \pi)^\top$$

Applying $(AB)^\top = B^\top A^\top$ on the left side,

$$\mathbf{P}^\top \pi^\top = \lambda \pi^\top$$

This is a right eigenvector equation for $\mathbf{P}^\top$. The reverse direction can be proved similarly by applying the transposing back. □

So: transpose $\mathbf{P}$, hand $\mathbf{P}^\top$ to NumPy, grab the $\lambda = 1$ eigenvector, normalize so entries sum to 1.

Why Does $\lambda = 1$ Always Exist?

Theorem

*If a stochastic matrix $\mathbf{P}$ is **regular** (i.e., some power $\mathbf{P}^k$ has all positive entries), then $\mathbf{P}$ has a unique equilibrium: there is only one eigenvalue 1. All other eigenvalues are smaller than 1.*

Why Does $\lambda = 1$ Always Exist?

Theorem

*If a stochastic matrix $\mathbf{P}$ is **regular** (i.e., some power $\mathbf{P}^k$ has all positive entries), then $\mathbf{P}$ has a unique equilibrium: there is only one eigenvalue 1. All other eigenvalues are smaller than 1.*

Note that our matrix has some values that are zero, meaning we can't reach every square in one step. But after enough steps, you can get from any square to any other. So $\mathbf{P}$ is regular, and the theorem applies.

Why Does $\lambda = 1$ Always Exist?

Theorem

*If a stochastic matrix $\mathbf{P}$ is **regular** (i.e., some power $\mathbf{P}^k$ has all positive entries), then $\mathbf{P}$ has a unique equilibrium: there is only one eigenvalue 1. All other eigenvalues are smaller than 1.*

Note that our matrix has some values that are zero, meaning we can't reach every square in one step. But after enough steps, you can get from any square to any other. So $\mathbf{P}$ is regular, and the theorem applies.

The actual proof is left as an exercise for the presentees.

We now have $\pi = (\pi_0, \pi_1, \dots, \pi_{39})$.

Results

We now have $\pi = (\pi_0, \pi_1, \dots, \pi_{39})$.

π_i : in the long run, a player spends a π_i proportion of their turns on square i .

We now have $\pi = (\pi_0, \pi_1, \dots, \pi_{39})$.

π_i : in the long run, a player spends a π_i proportion of their turns on square i .

To find the most visited squares, we sort the elements of π from largest to smallest.

Results

We now have $\pi = (\pi_0, \pi_1, \dots, \pi_{39})$.

π_i : in the long run, a player spends a π_i proportion of their turns on square i .

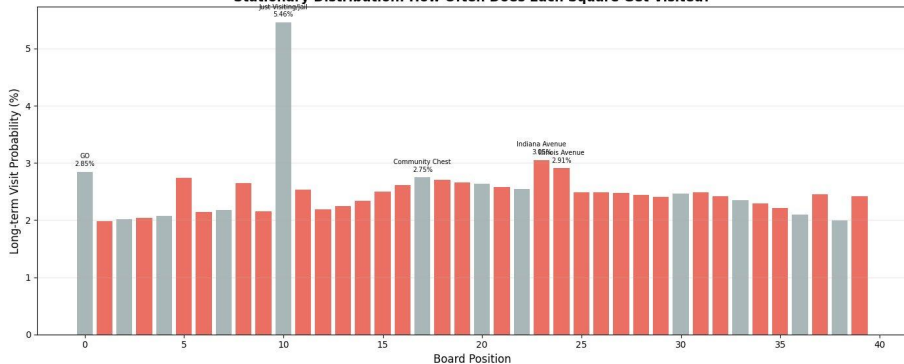
To find the most visited squares, we sort the elements of π from largest to smallest.

To find the best investment, we combine π_i with rent and cost:

$$\text{ROI}_i = \frac{\pi_i \times \text{rent}_i}{\text{cost}_i} \times 100\%$$

Final Distribution

Stationary Distribution: How Often Does Each Square Get Visited?



Most Visited Squares

Rank	Square	π_i (%)
1	Jail / Just Visiting	5.46
2	Indiana Avenue	3.05
3	Illinois Avenue	2.91
4	GO	2.85
5	Tennessee Avenue	2.70
6	New York Avenue	2.66

Most Visited Squares

Rank	Square	π_i (%)
1	Jail / Just Visiting	5.46
2	Indiana Avenue	3.05
3	Illinois Avenue	2.91
4	GO	2.85
5	Tennessee Avenue	2.70
6	New York Avenue	2.66

Jail is visited $\sim 2.2\times$ more than a uniform square (2.5%).

Most Visited Squares

Rank	Square	π_i (%)
1	Jail / Just Visiting	5.46
2	Indiana Avenue	3.05
3	Illinois Avenue	2.91
4	GO	2.85
5	Tennessee Avenue	2.70
6	New York Avenue	2.66

Jail is visited $\sim 2.2\times$ more than a uniform square (2.5%).

Three of the top 6 are **orange** or directly benefit from Jail traffic.

ROI: Which Properties Pay Off Fastest?

$$\text{ROI} = \frac{\pi_i \times \text{rent (hotel)}}{\text{total cost}} \times 100\%$$

ROI: Which Properties Pay Off Fastest?

$$\text{ROI} = \frac{\pi_i \times \text{rent (hotel)}}{\text{total cost}} \times 100\%$$

Color Group	Cost	Exp. Return	ROI%
Orange	\$2,060	\$77.19/turn	3.75
Light Blue	\$1,070	\$39.23/turn	3.67
Red	\$2,930	\$91.13/turn	3.11
Dark Blue	\$2,750	\$85.05/turn	3.09

ROI: Which Properties Pay Off Fastest?

$$\text{ROI} = \frac{\pi_i \times \text{rent (hotel)}}{\text{total cost}} \times 100\%$$

Color Group	Cost	Exp. Return	ROI%
Orange	\$2,060	\$77.19/turn	3.75
Light Blue	\$1,070	\$39.23/turn	3.67
Red	\$2,930	\$91.13/turn	3.11
Dark Blue	\$2,750	\$85.05/turn	3.09

Shout out to Brown, Green and Yellow for being bottom 3.

Some Interesting Observations

- Color sets right before a corner tend to outperform the ones right after. they catch the outflow from the previous high-traffic area. (i.e. Light Blue > Pink, Orange > Red, Yellow > Green, Dark Blue > Brown)

Some Interesting Observations

- Color sets right before a corner tend to outperform the ones right after. they catch the outflow from the previous high-traffic area. (i.e. Light Blue > Pink, Orange > Red, Yellow > Green, Dark Blue > Brown)
- The top 3 most visited purchasable squares are all within 14 squares of Jail.

Some Interesting Observations

- Color sets right before a corner tend to outperform the ones right after. they catch the outflow from the previous high-traffic area. (i.e. Light Blue > Pink, Orange > Red, Yellow > Green, Dark Blue > Brown)
- The top 3 most visited purchasable squares are all within 14 squares of Jail.
- Community Chest squares show up in the top 10 most visited, despite not being properties you can buy.

Some Interesting Observations

- Color sets right before a corner tend to outperform the ones right after. they catch the outflow from the previous high-traffic area. (i.e. Light Blue > Pink, Orange > Red, Yellow > Green, Dark Blue > Brown)
- The top 3 most visited purchasable squares are all within 14 squares of Jail.
- Community Chest squares show up in the top 10 most visited, despite not being properties you can buy.
- Dark Blue has the highest rent in the game, but only ranks 4th in ROI. Makes sense since rent doesn't matter if nobody lands there.

Why Orange

Buy the **orange** properties.

Properties	St. James, Tennessee, New York
Total cost	\$2,060
Expected return	\$77.19 per opponent turn
ROI	3.75%
Payback	~27 turns

Thank You

Rank	Color Group	ROI%
1	Orange	3.75
2	Light Blue	3.67
3	Red	3.11
4	Dark Blue	3.09
5	Pink	2.93
6	Yellow	2.82
7	Green	2.41
8	Brown	2.28